第三章 栈和队列 · 同步练习

> 配套：教材第 3 章（P38 ~ P57）
> 题型说明：**一 ~ 五 为教材 P56-57 同步训练原题**，**六、七 为补充题**。答案与解析统一放在最后，建议先独立完成再对照。
> 答题要求：算法设计题请用 C 语言写出**完整可编译**的函数或程序，并标注关键步骤。

一、填空题（教材 P56）

1. 栈的操作原则是 _____，队列的操作原则是 _____。
2. 循环队列用数组 `data[max]` 存放其元素值，已知其头、尾指针分别是 `front` 和 `rear`，则当前队列中元素的个数是 _____。
3. 假设以 `S` 和 `X` 分别表示进栈和出栈操作，则对输入序列 `a, b, c, d, e` 进行一系列栈操作 `SSXSXSSXXX` 之后，得到的输出序列为 _____。
4. 循环队列的优点是 _____。

二、选择题（教材 P56）

1. 循环队列是空队列的条件是（　　）。

A. `Q->rear == Q->front`
B. `(Q->rear+1) % maxsize == Q->front`
C. `Q->rear == 0`
D. `Q->front == 0`

2. 链栈与顺序栈相比，比较明显的优点是（　　）。

A. 插入操作更加方便
B. 删除操作更加方便
C. 不会出现下溢的情况
D. 不会出现上溢的情况

3. 设数组 `Data[n]` 作为循环队列 Q 的存储空间，`front` 为队头指针，`rear` 为队尾指针，则执行入队操作的语句为（　　）。

A. `Q->rear = (Q->rear+1) % (n+1)`
B. `Q->front = (Q->front+1) % n`
C. `Q->rear = (Q->rear+1) % n`
D. `Q->front = (Q->front+1) % (n+1)`

三、判断题（教材补充）

1. 栈和队列都是操作受限的线性表。（　　）
2. 顺序栈的栈顶指针 `top` 是一个指针变量，存放栈顶元素的地址。（　　）
3. 循环队列中，`front == rear` 说明队列一定是空的。（　　）

4. 链栈的栈顶在链表的表头，所以入栈操作相当于单链表的头插法。（　　）

5. 队列的删除操作在队尾进行。（　　）

6. 栈可以实现函数的递归调用，因为递归的返回顺序正好符合后进先出。（　　）

四、应用题（教材 P57）

1. 数据元素进栈次序为 `1, 2, 3`。进栈过程中允许出栈，请写出各种可能的出栈元素序列。

2. 分析下面程序段的功能，程序段中所调用的函数为顺序栈的基本操作。

```
void fun(SeqStack *s)
{
    int arr[max], n = 0, i;
    while (!StackEmpty(s))
    {
        arr[n++] = Pop(s);          // 教材版 Pop 直接返回栈顶元素
    }
    for (i = 0; i < n; i++)
        Push(s, arr[i]);
}
```

3. 分析下面程序段的功能，程序段中所调用的函数为顺序队列的基本操作。

```
void fun(CirQueue *Q, CirQueue *Q1, CirQueue *Q2)
{
    int e;
    InitQueue(Q1);
    InitQueue(Q2);
    while (!QueueEmpty(Q))
    {
        e = DeQueue(Q);
        if (e >= 0) EnQueue(Q1, e);
        else        EnQueue(Q2, e);
    }
}
```

五、算法设计题（教材 P57）

1. 回文是正读和反读都一样的序列，如 `"abcdcba"` 是回文，而 `"hello"` 不是回文。请编写一个算法，判断一个字符串是否是回文。**要求利用栈的知识来实现。**

2. 编写一个算法，计算循环链表中结点的个数。

六、补充题：程序阅读与改错

1. 阅读程序，写出输出结果。

```
SeqStack S;
int x;
InitStack(&S);
Push(&S, 1);
Push(&S, 2);
Pop(&S, &x);
Push(&S, 3);
Push(&S, 4);
Pop(&S, &x);
Pop(&S, &x);
printf("%d %d\n", x, S.top);
```

2. 阅读程序，写出 5 次入队后 `front`、`rear` 的值。

```
#define N 5
CirQueue Q;
int i, x;
InitQueue(&Q);
for (i = 1; i <= 5; i++) {
    if (!EnQueue(&Q, i)) printf("第 %d 次入队失败\n", i);
}
printf("front=%d, rear=%d\n", Q.front, Q.rear);
```

3. 找出下面函数中的错误并改正（共 3 处）。

```
int DeQueue(CirQueue *Q, int *x) {
    if ((Q->rear + 1) % QueueSize == Q->front) return 0;   // 判空
    *x = Q->data[Q->front];
    Q->front = Q->front + 1;
    return 1;
}
```

七、拓展提升（选做）

1. 用栈判断一个只含 `(`、`)`、`[`、`]`、`{`、`}` 的字符串是否括号匹配，不匹配时输出第一个出错位置。

2. 用栈把一个十进制正整数转换为八进制数。

3. 已知入栈序列为 `1, 2, 3, 4, 5`，判断序列 `4, 5, 3, 2, 1` 是否为合法的出栈序列，并说明理由。

4. 设有一个循环队列，容量为 `QueueSize`，请设计算法求队列中元素个数，**要求不使用 `front`、`rear` 之外的变量**（提示：只允许调用基本操作，考察对公式的理解）。

参考答案与解析

一、填空题

1. **后进先出（LIFO）；先进先出（FIFO）**

> 记忆：栈像弹夹，最后一颗压进去的最先射出；队列像食堂排队，先来的先打饭。

2. `(rear - front + max) % max`

> `rear` 可能已经绕到 `front` 前面，直接相减会得负数，所以要加一圈 `max` 再取模。
> 例：`max=5, front=4, rear=2` → `(2-4+5)%5 = 3`，队列里确实有 3 个元素。

3. `b c e d a`

> 逐步推演：
>
> | 操作 | 说明 | 栈内容（底→顶） | 输出 |
> | --- | --- | --- | --- |
> | S | a 进栈 | a | |
> | S | b 进栈 | a b | |
> | X | b 出栈 | a | b |
> | S | c 进栈 | a c | |
> | X | c 出栈 | a | c |
> | S | d 进栈 | a d | |
> | S | e 进栈 | a d e | |
> | X | e 出栈 | a d | e |
> | X | d 出栈 | a | d |
> | X | a 出栈 | 空 | a |
>
> 输出序列：**b c e d a**

4. **克服顺序队列的"假溢出"现象，充分利用存储空间**

> 普通顺序队列中，出队只让 `front` 后移，前面空出的位置再也用不上；`rear` 到数组末尾时明明还有空位却无法入队。循环队列让首尾相接，空位可以重复利用。

二、选择题

1. A

> 队空：`front == rear`（两人站在同一格）；队满：`(rear+1) % maxsize == front`（`rear` 再走一格就撞上 `front`）。B 是队满条件，C、D 与循环队列无关。

2. D

> 顺序栈容量固定，栈满再入栈就是**上溢**；链栈按需申请结点，只要内存还有就不会"栈满"。
> 注意：C 不对——链栈空栈时出栈照样是**下溢**，只是判断条件变成了 `S == NULL`。

3. C

> 入队只动 `rear`，且数组大小是 `n`，所以是 `Q->rear = (Q->rear+1) % n`。A 用了 `n+1`，会越界；B、D 动的是 `front`，那是出队。

三、判断题

题号	答案	解析
1	√	两者都是线性表，只是限制了插入 / 删除的位置
2	×	`top` 是**整型下标**，不是指针；用 `data[top]` 取栈顶
3	×	`front == rear` 也可能是**队满**（牺牲一个单元后，绕一圈回来会重合）
4	√	链栈的栈顶就是链表第一个结点，入栈即头插
5	×	队列在**队头**删除、**队尾**插入
6	√	递归的调用与返回正好是"后进先出"，系统用栈保存返回地址

四、应用题

1. 可能的出栈序列共 **5 种**：

```
1 2 3      1 3 2      2 1 3      2 3 1      3 2 1
```

不可能出现的是 `3 1 2`。

> 推演：要 3 先出栈，必须先把 1、2、3 依次压栈；3 出栈后栈内为 `1 2`，栈顶是 2，只能先出 2 再出 1，所以不可能得到 `3 1 2`。
> **通用结论：** 出栈序列中若出现 `i < j < k` 且 `k` 最先出，则 `j` 必须比 `i` 先出。

2. 功能：把栈 `s` 中的元素逆置。

> 分析：
>
> - 第一个 `while`：反复出栈，把元素依次存入 `arr[0]、arr[1]…`。出栈顺序是**原栈顶 → 原栈底**，所以 `arr` 中存的是"从顶到底"；
> - 第二个 `for`：按 `arr[0] → arr[n-1]` 的顺序重新压栈，于是**原栈顶被压到最底下**，原栈底变成了新栈顶；
> - 结果：栈中元素个数不变，但**顺序完全颠倒**（栈顶与栈底互换）。
>
> 举例：原栈从底到顶 `1 2 3` → 处理后从底到顶 `3 2 1`。
> 该算法时间 O(n)、空间 O(n)。

3. 功能：把队列 `Q` 中的元素按"非负 / 负数"拆分成两个队列 `Q1`、`Q2`。

> 分析：
>
> - `Q1`、`Q2` 先初始化为空队列；
> - 循环中每次从 `Q` 出队一个元素 `e`：`e >= 0` 入 `Q1`，`e < 0` 入 `Q2`；
> - 循环结束时 `Q` 已空，元素被分成两组，且**各自保持了原来的相对次序**（因为出队顺序就是原顺序，入队也是尾插）。
>
> 举例：`Q = 3, -1, 0, -5, 8` → `Q1 = 3, 0, 8`，`Q2 = -1, -5`。

五、算法设计题

1. 判断回文（用栈）

思路： 回文正读反读相同，而"反读"正是栈的特长——全部入栈后逐个出栈，就得到了逆序串，再与原串逐字符比较。

```
#include <stdio.h>
#include <string.h>

#define StackSize 100
typedef char DataType;

typedef struct {
    DataType data[StackSize];
    int top;
} SeqStack;

void InitStack(SeqStack *S)  { S->top = -1; }
int  StackEmpty(SeqStack *S) { return S->top == -1; }

int Push(SeqStack *S, DataType x) {
    if (S->top == StackSize - 1) return 0;      // 栈满
    S->data[++S->top] = x;                      // 先挪 top，再放
    return 1;
}

int Pop(SeqStack *S, DataType *x) {
    if (StackEmpty(S)) return 0;                // 空栈，下溢
    *x = S->data[S->top];                       // 先取
    S->top--;                                   // 再挪
    return 1;
}

// 回文判断：是回文返回 1，否则返回 0
int IsPalindrome(char *s) {
    SeqStack st;
    char ch;
    int i, n = (int)strlen(s);

    InitStack(&st);

    for (i = 0; i < n; i++) {                   // ① 全部入栈
        Push(&st, s[i]);
    }
    for (i = 0; i < n; i++) {                   // ② 出栈得到逆序，与原串比较
        Pop(&st, &ch);
        if (ch != s[i]) return 0;               // 有一个不同就不是回文
    }
    return 1;
}

int main() {
```

```
    char str[100];
    printf("请输入字符串：");
    scanf("%s", str);                              // 数组名不加 &
    printf("%s\n", IsPalindrome(str) ? "是回文" : "不是回文");
    return 0;
}
```

> **评分要点：** ① 栈的初始化与入栈正确；② 出栈顺序与比较方向正确；③ 提前返回 `0` 的短路逻辑；④ 边界（空串、单字符）返回 `1`。
> **时间 O(n)，空间 O(n)。**
> **变式：** 也可只把前一半入栈，再与后一半比较，空间可减半。

2. 求循环链表中结点的个数

思路： 循环链表没有 `NULL` 结尾，**回到起点就是走完一圈。**从第一个结点出发，走一步计数加一，直到指针重新等于起点。

```
#include <stdio.h>
#include <stdlib.h>

typedef struct LNode {
    int data;
    struct LNode *next;
} LNode, *LinkList;

// L 指向循环链表中的第一个结点（不带头结点的写法）
int CountNode(LinkList L) {
    LNode *p;
    int n = 0;

    if (L == NULL) return 0;        // 空表

    p = L;
    do {
        n++;                        // ① 计数
        p = p->next;                // ② 往后走
    } while (p != L);               // ③ 回到起点就停

    return n;
}

int main() {
    // 手工构造循环链表：1 -> 2 -> 3 -> 回到 1
    LNode a = {1, NULL}, b = {2, NULL}, c = {3, NULL};
    a.next = &b;  b.next = &c;  c.next = &a;

    printf("结点个数 = %d\n", CountNode(&a));   // 输出 3
    return 0;
}
```

> **评分要点：** ① 用 `do-while`（先走一步再判断）或"`p = L->next` 起、`while (p != L)`"两种写法；② 必须处理空表；③ 若链表**带头结点**，则从 `L->next` 开始、`while (p != L)` 结束，计数不含头结点。
> **时间 O(n)，空间 O(1)。**

六、补充题答案

1. 输出：`3 0`

> 推演：
>
> | 操作 | 栈（底→顶） | `x` |
> |---|---|---|
> | push 1 | 1 | - |
> | push 2 | 1 2 | - |
> | pop | 1 | 2 |
> | push 3 | 1 3 | 2 |
> | push 4 | 1 3 4 | 2 |
> | pop | 1 3 | 4 |
> | pop | 1 | 3 |
>
> 最后 `x = 3`，`top = 0`（栈里剩 `1` 一个元素）。

2. 输出：

```
第 5 次入队失败
front=0, rear=4
```

> `N = 5`，按"牺牲一个单元"的约定，最多只能存 **4 个**元素。
>
> | 次数 | 入队前 `(rear+1)%5` | 结果 | 入队后 `rear` |
> |---|---|---|---|
> | 1 | `(0+1)%5 = 1 ≠ 0` | 成功 | 1 |
> | 2 | `(1+1)%5 = 2 ≠ 0` | 成功 | 2 |
> | 3 | `(2+1)%5 = 3 ≠ 0` | 成功 | 3 |
> | 4 | `(3+1)%5 = 4 ≠ 0` | 成功 | 4 |
> | 5 | `(4+1)%5 = 0 == front` | **队满，失败** | 仍为 4 |
>
> 本题要点：**容量为 N 的循环队列最多存 N-1 个元素；队满时 `rear` 不再移动。**

3. 三处错误

#	错误位置	问题	改正
1	`if ((Q->rear+1) % QueueSize == Q->front) return 0;`	这是**判满**条件，写成了判空，导致队满时反而能出队	`if (Q->front == Q->rear) return 0;`
2	`Q->front = Q->front + 1;`	没有取模，`front` 会越界	`Q->front = (Q->front + 1) % QueueSize;`
3	缺 `QueueSize` 的定义 / 参数 `x` 未判空	若调用方传 `NULL` 会崩溃；且函数未说明空队列时 `x` 不变	补 `#define QueueSize 50`，并在函数入口 `if (x == NULL) return 0;`

七、拓展提升参考答案

1. 多类型括号匹配（要点片段）

```
int Match(char *s) {
    SeqStack st;
    char ch;
    InitStack(&st);
    for (; *s != '\0'; s++) {
        if (*s == '(' || *s == '[' || *s == '{') {
            Push(&st, *s);                          // 左括号入栈
        } else if (*s == ')' || *s == ']' || *s == '}') {
            if (StackEmpty(&st)) return 0;          // 右括号多余
            Pop(&st, &ch);
            if ((*s == ')' && ch != '(') ||
                (*s == ']' && ch != '[') ||
                (*s == '}' && ch != '{')) {
                return 0;                           // 类型不配对
            }
        }
        /* 其他字符忽略 */
    }
    return StackEmpty(&st);                         // 左括号必须全部配对完
}
```

> 输出出错位置：把 `return 0` 改成 `printf("第 %d 个字符出错\n", s - start + 1); return 0;`。

2. 十进制转八进制

```
void DecToOct(int n) {
    SeqStack st;
    InitStack(&st);
    if (n == 0) { printf("0\n"); return; }
    while (n > 0) {
        Push(&st, n % 8);          // 余数入栈（低位先进）
        n = n / 8;
    }
    while (!StackEmpty(&st)) {     // 出栈即逆序，得到高位在前
```

```
        int x;
        Pop(&st, &x);
        printf("%d", x);
    }
    printf("\n");
}
```

> 例：`100` → 余数依次 `4, 4, 1` → 出栈 `1 4 4` → `144`（八进制）。

3. 合法，`4 5 3 2 1` 是合法出栈序列

> 推演：`1 2 3 4` 依次入栈 → 出 `4` → `5` 入栈 → 出 `5` → 栈内剩 `1 2 3` → 依次出 `3 2 1`。
> 得到的正是 `4 5 3 2 1`，所以合法。

4. 求循环队列元素个数（只用基本操作）

```
int CountQueue(CirQueue *Q) {
    CirQueue tmp;
    int n = 0, x;
    InitQueue(&tmp);
    while (!QueueEmpty(Q)) {          // 边出队边暂存到 tmp
        DeQueue(Q, &x);
        EnQueue(&tmp, x);
        n++;
    }
    while (!QueueEmpty(&tmp)) {       // 再还回去，保持原队列不变
        DeQueue(&tmp, &x);
        EnQueue(Q, x);
    }
    return n;
}
```

> 代价：时间 O(n)，且需要额外一个队列（空间 O(n)）。实际工程中直接用公式 `(rear-front+N)%N`，O(1)。
> 本题目的在于理解"循环队列元素个数"的两种求法及其差异。